

Guideline for Scrum Assignments Document Submission

Documents should specify team identification (name and or designation) on

- front page and in
- header on all subsequent numbered pages

PDF format preferred.

The documents you create for the scrum assignments will form part of your **Final Submission**, which will include final versions of

- Requirements (Product Backlog)
- Design Document
- Testing Document

also

- well-commented source code with
- a description of the project directory structure and class files

The Requirements, Design and Testing Document is **ONE** document.

The front matter of the document will contain:

- Cover page (title, authors, date, course, etc.)
- Abstract (1-2 paragraphs)
- Table of Content (section no., title and page no.)
- List of Figures (figure no., title and page no.)
- List of Tables (table no., title and page no.)
- Abbreviations & Definitions

The main body of the document will contain:

- Requirements (Product Backlog)
- Design Document
- Testing Document
- Sprint Backlog

The back matter of the document will contain:

- References (in one known format, e.g., IEEE)
- Appendices

For Scrum Assignment X you should include the following in the submitted document:

Requirements Version X:

Product backlog, with items subdivided further into tasks and relative sizing. Showing completed and remaining items.

Design Document Version X:

The Design Document for Sprint X will add on to the previous design document; the design document should cover everything designed and implemented for the app since the beginning. By the end of the project the design document will become part of your final submission covering the design of your project.

Testing Document Version X:

Similar to the design document, the testing document will cover all the tests done since the beginning of the project. If any test fails before, it is expected to be solved and documented in the next version of the testing document.

Sprint X Backlog:

Sprint X backlog will show your planning for the next coming sprint, with items explained in more details and divided into smaller tasks.

When meeting with the scrum master at the end of each sprint and before the next sprint you will have to **demonstrate** your “Done” code against the last sprint backlog. For every item on your sprint backlog you need to show a small demo of it working. If not all the items on your sprint backlog are “Done” then the scrum master will discuss with the team, why did that happen and how to plan better for next upcoming sprints.

The scrum master will also discuss the sprint backlog of the next sprint and give his/her comments to the team.

This demonstration is part of your Scrum Assignments, and any adjustments recommended by the scrum master should be adjusted as well in the PB, design and testing document as needed.

Report Formatting

The whole document should be in ONE format, pretty please.

- Same margin, list and paragraph spacing
- Same font, size, spacing etc...
- Page number that increment throughout the whole ONE document
- Make sure there is no vocabulary or punctuation mistakes
- The table of content and the two lists are perfectly synced with the page numbers
- Every figure or table has a label and a title

COEN/ELEC 390 Scrum Assignment Guide

Team Name

Member 1 ID

Member 2 ID

Member 3 ID

Abstract— The following is a guide that explains the content of the requirement, design and testing document. Each project is different, this guide is a guide on how to document the requirements, design and testing for an android application, but based on your project you might add a new section that you believe it is important.

This document is intended to describe the requirements of the project as well as the design and testing of the project.

The Requirements Document (PB) is key to communication between the managers and his customers as it describes what the project must do. The managers must agree on the requirements specified in the requirements document for successful project development and delivery. The requirements document is also key in designing the project.

The Design Document is a crucial element of a successful project as it is key to communication between the developers and their managers. It acts as an agreement between these groups on how to organize and achieve the functionality that was defined in the requirements specification. It documents and tracks the necessary information required to effectively define architecture and system design in order to give the development team guidance on the architecture of system to be developed.

The Design Document will be used as a template from which the developers can implement code independently and in parallel while maintaining a common vision of how their code will interact with other parts of the program and what the end result will be.

It should be an evolving “living document” that is updated and fleshed out as the project implementation proceeds. For long-term projects this is critical to enable new team members to quickly learn how the project works. (It is easier to read diagrams and text than source code). It also allows the development team (or a different group) to revisit the project, perhaps long after it is done, to fix a bug or add new features.

The document must be sufficiently detailed so that all the developers involved have the same idea of how the system should work.

Software testing is an important technique for assessing the quality of a software product. Software testing is the process of analyzing a software item to detect the differences between existing and required conditions (that is, bugs) and to evaluate the features of the software item.

Software testing is one of the “verification and validation,” or V&V, software practices.

Verification: Are we building the product right?

Through verification, we make sure the product behaves the way we want it to.

Validation: Are we building the right product?

Through validation, we check to make sure that somewhere in the process a mistake hasn’t been made such that the product built is not what the customer asked for; validation always involves comparison against requirements.

Note: This is the bare minimum sections required in the design document. More sections can be added if necessary.

Table of Contents

1	Introduction.....	1
1.1	Definitions, Acronyms, and Abbreviations.....	1
2	Requirements: Product Backlog	2
3	Design	3
3.1	Android Application Wireframes	3
3.2	System Architecture	3
3.3	Hardware Architecture.....	4
3.4	Software Architecture	4
3.5	Use Cases and Sequence Diagrams.....	4
4	Testing	4
4.1	Test Plan 1: Story Title.....	5
4.1.1	Summary	5
4.1.2	Test Cases.....	5
4.2	Test Plan 2: Story Title.....	6
4.2.1	Summary	6
4.2.2	Test Cases.....	6
5	Sprint X Backlog	7
6	References	8
	Appendix A:	9

List of Figures

Figure 1	An example diagram showing the wireframe of a mobile app.	3
Figure 2	A simple architecture diagram for an app that sends a message, saves the message in a cloud database and acknowledges that message is received.	3
Figure 3	Sequence Diagram for when "Play Round" button is pressed.	4

List of Tables

Table 1	Definition and Abbreviation Example	1
---------	---	---

1 Introduction

This section should:

- o Identify the product to be developed
- o Introduce the general functionality highlighting the most important features without getting into too much details.
- o Define the relevant benefits, objectives and goals as precisely as possible.
- o Mention who the potential user is and how will they benefit from the product in general.

1.1 Definitions, Acronyms, and Abbreviations

The following table details the specific terms and abbreviations used throughout this report.

Table 1 Definition and Abbreviation Example

Abbreviation	Term	Meaning
API	Application Program Interface	Set of protocols on the software to specify how components should interact.
GPS	Global Positioning System	Satellite-based positioning system. Provides location based on data from satellites.
GUI	Graphical User Interface	Type of user interface that allows the user to interact with graphical icons such as buttons on screen or text fields on screen.

2 Requirements: Product Backlog

Story ID	Story Title	Card	Story Points	Sprint	Status	Conversation	Confirmation	Story Tag
BK-1	Scalability	<i>As a power user, I want the ability to select the files and folders to backup so that I do not backup files and folders that need not be backed up.</i>	9	Sprint 1		Having this ability will ensure that users do not backup files and folder that are not needed. Also, it will decrease the time and space required for the backup process.	1. Can the user select folders to be included in backup 2. Can the user select only particular files of a folder to be backed up	
BK-2	Functionality	<i>As a power user, I want the backup destination to be configurable so that I can specify the backup destination based on various factors.</i>	7	Sprint 1		This will allow the user to select a backup destination based on various factors like space available, etc. Provide ability to specify all available destinations like SD card, network, etc.	1. Can the user specify backup destination.	

3 Design

3.1 Android Application Wireframes



Figure 1 An example diagram showing the wireframe of a mobile app.

3.2 System Architecture

This section should describe the architecture of the solution that is being implemented with high level data flows described to set the context of the system. Describe the various components of the system (hardware, software, etc.), how the various components interact with each other and how data (if any) is correctly passed between the various components of the system.

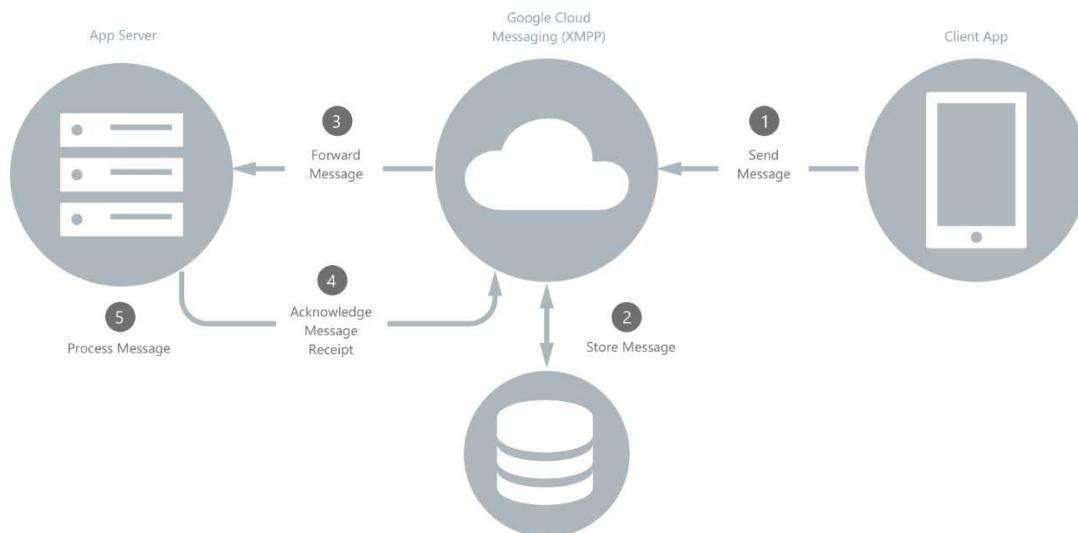


Figure 2 A simple architecture diagram for an app that sends a message, saves the message in a cloud database and acknowledges that message is received. [1]

3.3 Hardware Architecture

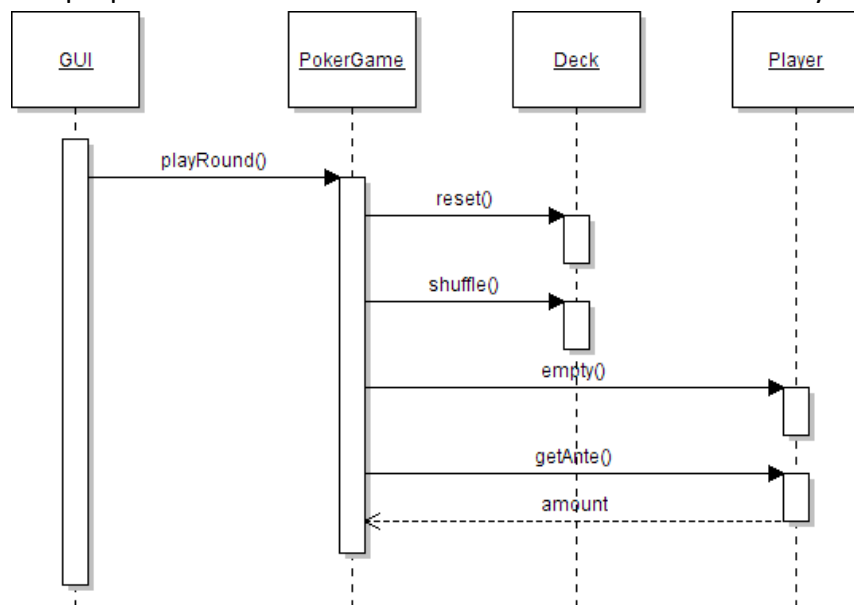
This section provides description of the sensors used in the system outlining the data gathered by the sensors and passed on to other components of the system

3.4 Software Architecture

This section is the “big picture” view of the software system and allows you to describe the structure of the software. The purpose of this section is to summarize the software architecture of the software system. This should be represented in the form of component diagrams showing all major components of the software system and their interactions accompanied by a short narrative explaining what the diagram is showing.

3.5 Use Cases and Sequence Diagrams

The purpose of this section is to describe the execution of the system in detail. This section should



identify multiple use cases that describe the complete working of the system and provide sequence diagrams for each use case. A use case is a list of actions or event steps, typically defining the interactions between a role (known in the Unified Modeling Language as an actor) and a system, to achieve a goal. The actor can be a human or other external system.

Figure 3 Sequence Diagram for when "Play Round" button is pressed.

4 Testing

We want to write test cases that have a high likelihood of uncovering the faults that are the most likely to be observed as a failure in normal use. There are two basic classes of software testing:

- Black box testing (also called functional testing) is testing that ignores the internal mechanism of a system or component and focuses solely on the outputs generated in response to selected inputs and execution conditions.
- White box testing (also called structural testing and glass box testing) is testing that takes into account the internal mechanism of a system or component.

For your COEN/ELEC 390 projects, you will be documenting your Black box testing only.

That does not mean that you will not do any White Box testing, to verify and validate your app you might need to have unit testing or integration testing. However, your document has to reflect test cases in the perspective of the user; testing the high-level design, hence Black Box testing.

The Testing Document should have test plans that are needed to cover all your requirements (specified in the product backlog). There should be test plan for every story specified in the PB and each test plan will have multiple test cases. Before every test plan write a summary of what you are trying to detect. Don't write a summary of the test cases, for example you can specify the requirements you are testing and what are you expecting to happen. The test cases should be presented in a table (format specified below). Specifying the preconditions of a test case, the behavior the tester will do and the expected result. After running the test, you specify the actual result.

Tips: When writing your test cases, think about how can you crash your app, and test that. Write your test cases as a person who is not the programmer of the code and does not know anything about the structure of the code.

- Enter a very large number as input for age
- Enter numbers for weight and height that are not humanly possible

4.1 Test Plan 1: Story Title

4.1.1 Summary

Requirement ID: BK-1 (this should trace to a requirement specified in BP).

Provide a brief summary of the test plan, i.e. what you are trying to detect. Do not write a summary of the test cases, instead briefly describe the requirement you are trying to test and what you are expecting to happen.

4.1.2 Test Cases

*This section describes in detail all the tests to be performed for this requirement in the following tabular format. **Each requirement will have multiple test cases. There should be one table for each test case.***

Test Case: 1.1		
Pre-Condition: <i><describe pre-conditions needed to perform the test></i>		
Steps:	Expected Results	Actual Results

1.
2.
3.
<i>n.</i>
Result: Pass/Fail

Test Case: 1.2		
Pre-Condition: <i><describe pre-conditions needed to perform the test></i>		
Steps:	Expected Results	Actual Results
1.		
2.		
3.		
<i>n.</i>		
Result: Pass/Fail		

4.2 Test Plan 2: Story Title

4.2.1 Summary

4.2.2 Test Cases

5 Sprint X Backlog

Sprint X Goal(s): <Describe what you would like to achieve at the end of sprint 1>

Story ID	Task ID	Task Title	Task Description	Ideal Hours	Status	Comments
BK-1	BK-1.1	File/Folder Selection UI	Create a UI so user can select the files and folders that can be selectively backed up. Provide ability to select complete folder or individual files in the folder	<= 5 hrs	Planned	
BK-1	BK-1.2	Backup task	<i>Backup only files that re selected by the user.</i>	<= 5 hrs	Planned	
BK-2	BK-2.1		<Epic BK-2, Task 1 description>	<= 5 hrs	Planned	
	BK-2.2		<Epic BK-2, Task 2 description>	<= 5 hrs	Planned	
	BK-2.3		<Epic BK-2, Task 3 description>	<= 5 hrs	Planned	

6 References

[1] "Google Cloud Messaging - Xamarin", *Developer.xamarin.com*, 2017. [Online]. Available: https://developer.xamarin.com/guides/android/application_fundamentals/notifications/google-cloud-messaging/. [Accessed: 31- Aug- 2017].

Appendix A:

In the appendix, you can include information that is not important for everyone to read (otherwise it should be mentioned in the main document), but someone would be interested in reading it.

For example, screenshots of the mobile app would go into the appendix.